Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Simulación
Ciclo	5
Nombre del Estudiante	Miguel Veintimilla
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	02
Título de la Práctica	Comparación de modelos determinísticos, estocásticos, discretos y continuos
Nombre del Docente	José O. Guamán Q.
Fecha	08 de Octubre de 2026 09 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica:

Comprender el proceso de construcción de un modelo matemático y analizar su comportamiento mediante Python.

1. Identificar variables, parámetros y relaciones matemáticas de un sistema.
2. Implementar un modelo matemático en Python.
3. Analizar gráficamente la influencia de los parámetros

3. Materiales y reactivos:

NumPy.

Matplotlib

4. Equipos y herramientas

- Computador.
- Python 3.
- Java

- Editor de código

5. Procedimiento / Metodología

Para la resolución de esta práctica y la simulación del comportamiento atmosférico, se aplicó una metodología de desarrollo de software basada en el patrón de arquitectura Modelo-Vista-Controlador (MVC), implementado en el lenguaje de programación Python.

El procedimiento constó de las siguientes fases:

1. Análisis del Modelo Matemático: Se identificaron las variables principales (Humedad, Nubosidad y Temperatura) y se analizó la ecuación principal del índice de lluvia: $I = 0.5H + 0.3N + 0.2Tf$. Se estructuraron las reglas condicionales para determinar el factor de temperatura (T_f) y los estados del clima (Sin lluvia, Baja posibilidad, Lluvia probable, Lluvia).

2. Diseño de la Arquitectura (MVC):

Para garantizar las buenas prácticas de programación, el código se dividió en tres componentes independientes:

- **Modelo (**modelo.py**):** Se encapsuló toda la lógica matemática. Esta clase se encargó de normalizar los porcentajes de entrada, aplicar las tablas de reglas climáticas y calcular el índice final. Se parametrizaron los pesos de la fórmula ($0.5, 0.3, 0.2$) para permitir ajustes dinámicos.
- **Vista (**vista.py**):** Se diseñó la interfaz de salida. Utilizando la librería Matplotlib, se programó el renderizado de la gráfica comparativa y la impresión tabulada de los datos en la terminal.
- **Controlador (**controlador.py**):** Se programó el flujo de la simulación, iterando sobre los vectores de tiempo y datos meteorológicos, enviándolos al modelo para su cálculo y posteriormente a la vista para su representación.

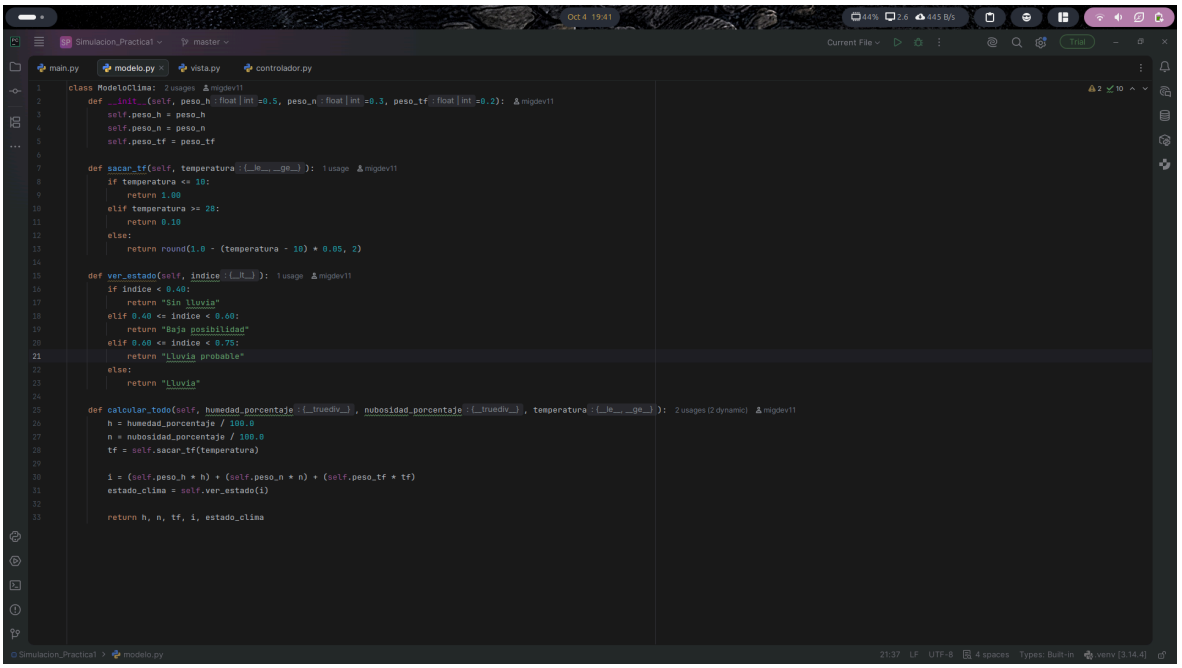
3. Simulación Computacional y Ajuste: Mediante el archivo de ejecución principal (main.py), se inicializó el simulador recorriendo las horas desde las 06:00 hasta las 22:00. Para cumplir con el análisis del sistema, se ejecutaron dos instancias del modelo simultáneamente:

- **Modelo Original:** Con los parámetros base de la guía ($H = 0.5, N = 0.3$).
- **Modelo Ajustado:** Modificando los pesos matemáticos ($H = 0.3, N = 0.5$) para evaluar la sensibilidad del sistema.

Finalmente, los resultados de ambas simulaciones fueron recopilados en listas y graficados en un único plano cartesiano, incorporando una línea de límite estático (0.75) para contrastar e identificar visualmente las horas exactas en las que el modelo predice lluvia.

6. Resultados esperados:

- Modelo implementado correctamente.



```python
class ModeloClima:
    def __init__(self, peso_h=0.5, peso_n=0.3, peso_tf=0.2):
        self.peso_h = peso_h
        self.peso_n = peso_n
        self.peso_tf = peso_tf

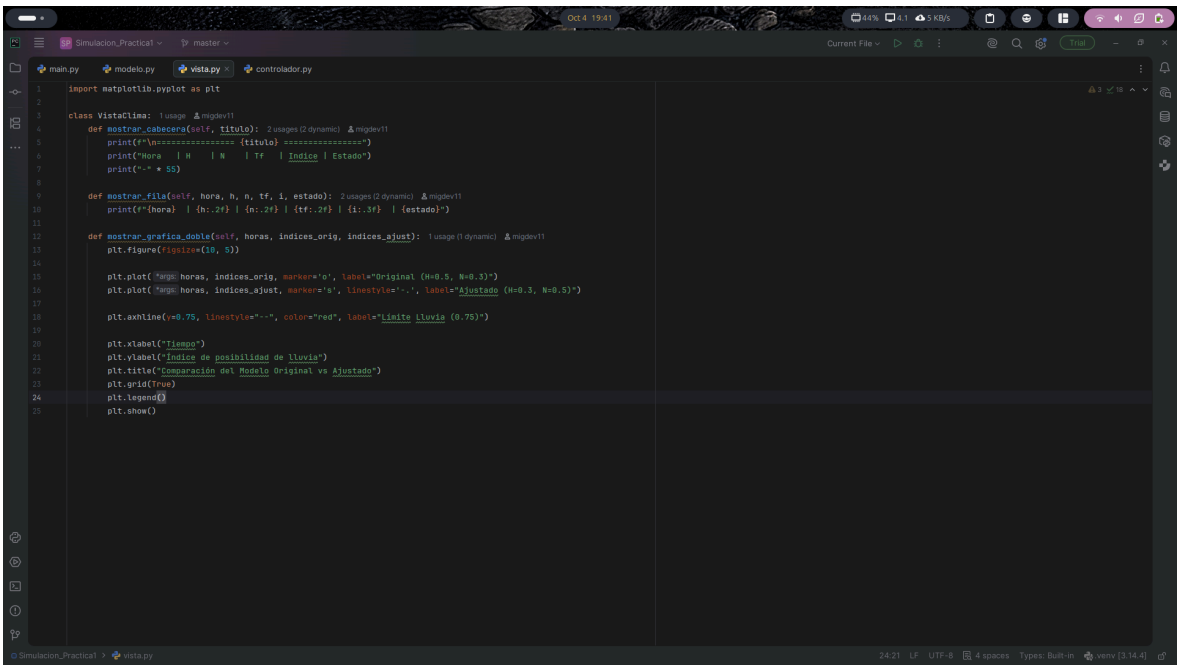
    def sacar_tf(self, temperatura):
        if temperatura <= 10:
            return 1.00
        elif temperatura >= 28:
            return 0.10
        else:
            return round(1.0 - (temperatura - 10) * 0.05, 2)

    def ver_estado(self, indice):
        if indice < 0.40:
            return "Sin lluvia"
        elif 0.40 <= indice < 0.60:
            return "Baja posibilidad"
        elif 0.60 <= indice < 0.75:
            return "Lluvia probable"
        else:
            return "Lluvia"

    def calcular_todo(self, humedad_porcentaje, nubosidad_porcentaje, temperatura):
        h = humedad_porcentaje / 100.0
        n = nubosidad_porcentaje / 100.0
        tf = self.sacar_tf(temperatura)

        i = (self.peso_h * h) + (self.peso_n * n) + (self.peso_tf * tf)
        estado_clima = self.ver_estado(i)

        return h, n, tf, i, estado_clima
```



```python
import matplotlib.pyplot as plt

class VistaClima:
    def mostrar_cabecera(self, titulo):
        print(f"\n=============== {titulo} ===============")
        print("Hora   | H    | N    | Tf   | Indice | Estado")
        print("-" * 55)

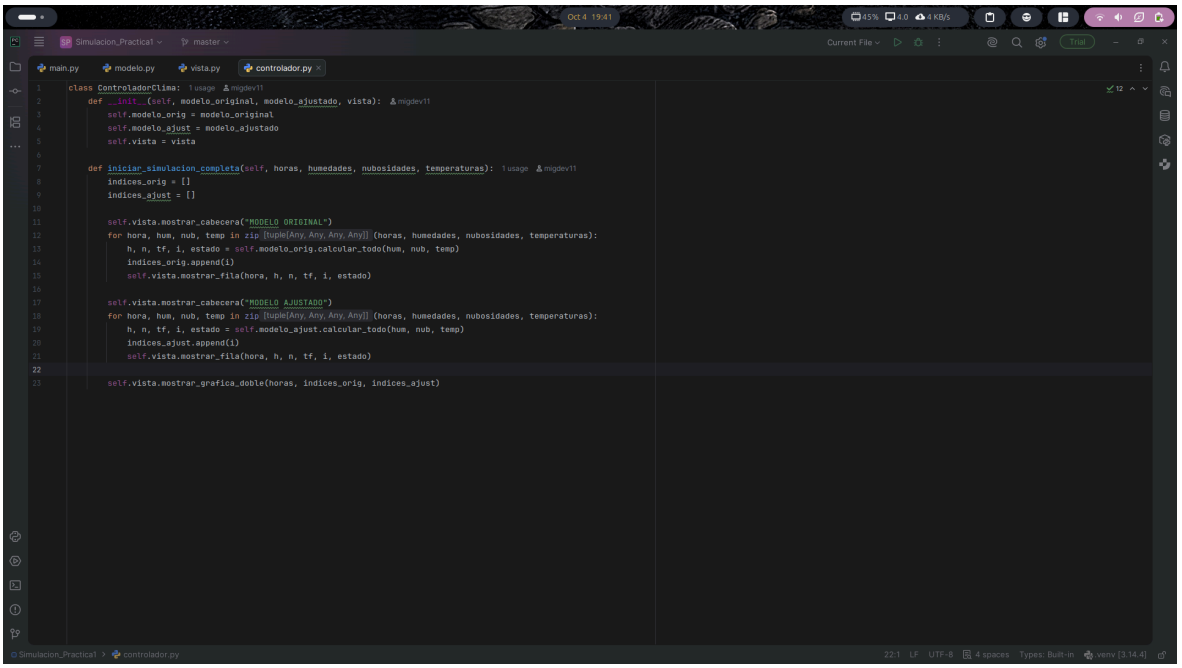
    def mostrar_fila(self, hora, h, n, tf, i, estado):
        print(f"{hora}  | {h:.2f} | {n:.2f} | {tf:.2f} | {i:.3f}  | {estado}")

    def mostrar_grafica_doble(self, horas, indices_orig, indices_ajust):
        plt.figure(figsize=(10, 5))

        plt.plot(*args: horas, indices_orig, marker='o', label="Original (H=0.5, N=0.3)")
        plt.plot(*args: horas, indices_ajust, marker='s', linestyle='--', label="Ajustado (H=0.3, N=0.5)")

        plt.axhline(y=0.75, linestyle="--", color="red", label="Limite Lluvia (0.75)")

        plt.xlabel("Tiempo")
        plt.ylabel("Indice de posibilidad de lluvia")
        plt.title("Comparación del Modelo Original vs Ajustado")
        plt.grid(True)
        plt.legend()
        plt.show()
```



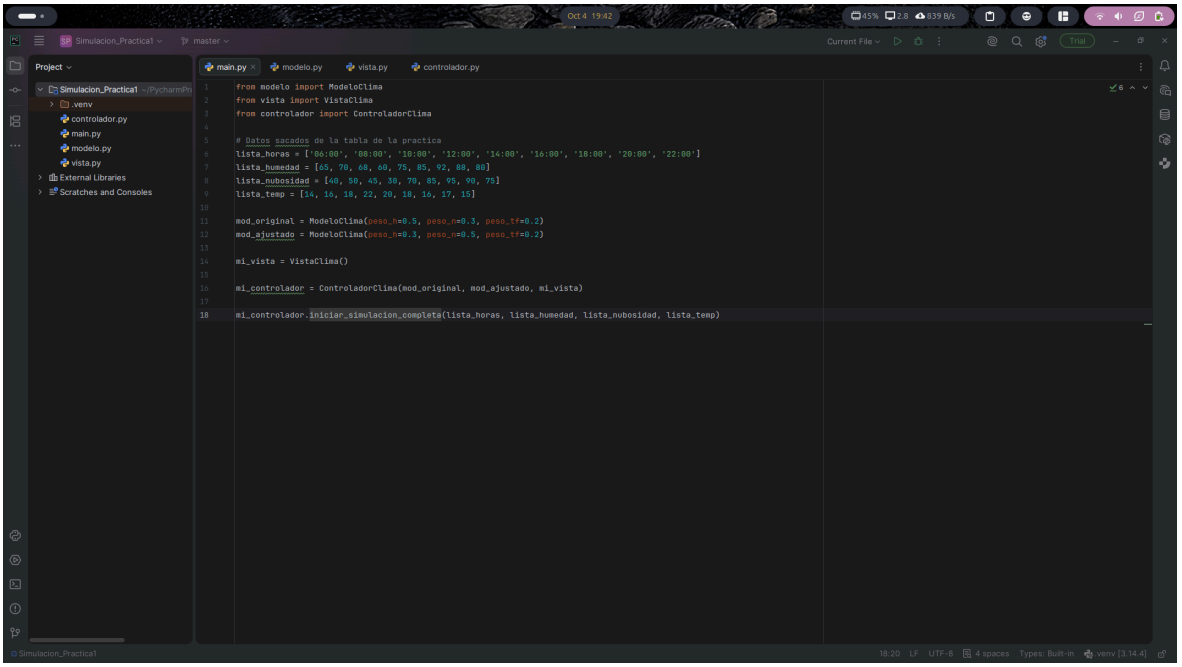
```python
class ControladorClima:  1 usage  migdev11
    def __init__(self, modelo_original, modelo_ajustado, vista):  migdev11
        self.modelo_orig = modelo_original
        self.modelo_ajust = modelo_ajustado
        self.vista = vista

    def iniciar_simulacion_completa(self, horas, humedades, nubosidades, temperaturas):  1 usage  migdev11
        indices_orig = []
        indices_ajust = []

        self.vista.mostrar_cabecera("MODELO ORIGINAL")
        for hora, hum, nub, temp in zip [tuple[Any, Any, Any, Any]] (horas, humedades, nubosidades, temperaturas):
            h, n, tf, i, estado = self.modelo_orig.calcular_todo(hum, nub, temp)
            indices_orig.append(i)
            self.vista.mostrar_fila(hora, h, n, tf, i, estado)

        self.vista.mostrar_cabecera("MODELO AJUSTADO")
        for hora, hum, nub, temp in zip [tuple[Any, Any, Any, Any]] (horas, humedades, nubosidades, temperaturas):
            h, n, tf, i, estado = self.modelo_ajust.calcular_todo(hum, nub, temp)
            indices_ajust.append(i)
            self.vista.mostrar_fila(hora, h, n, tf, i, estado)

        self.vista.mostrar_grafica_doble(horas, indices_orig, indices_ajust)
```



```python
from modelo import ModeloClima
from vista import VistaClima
from controlador import ControladorClima

# Datos sacados de la tabla de la practica
lista_horas = ['06:00', '08:00', '10:00', '12:00', '14:00', '16:00', '18:00', '20:00', '22:00']
lista_humedad = [65, 70, 68, 60, 75, 85, 92, 88, 80]
lista_nubosidad = [40, 50, 45, 30, 70, 85, 95, 90, 75]
lista_temp = [14, 16, 18, 22, 20, 18, 16, 17, 15]

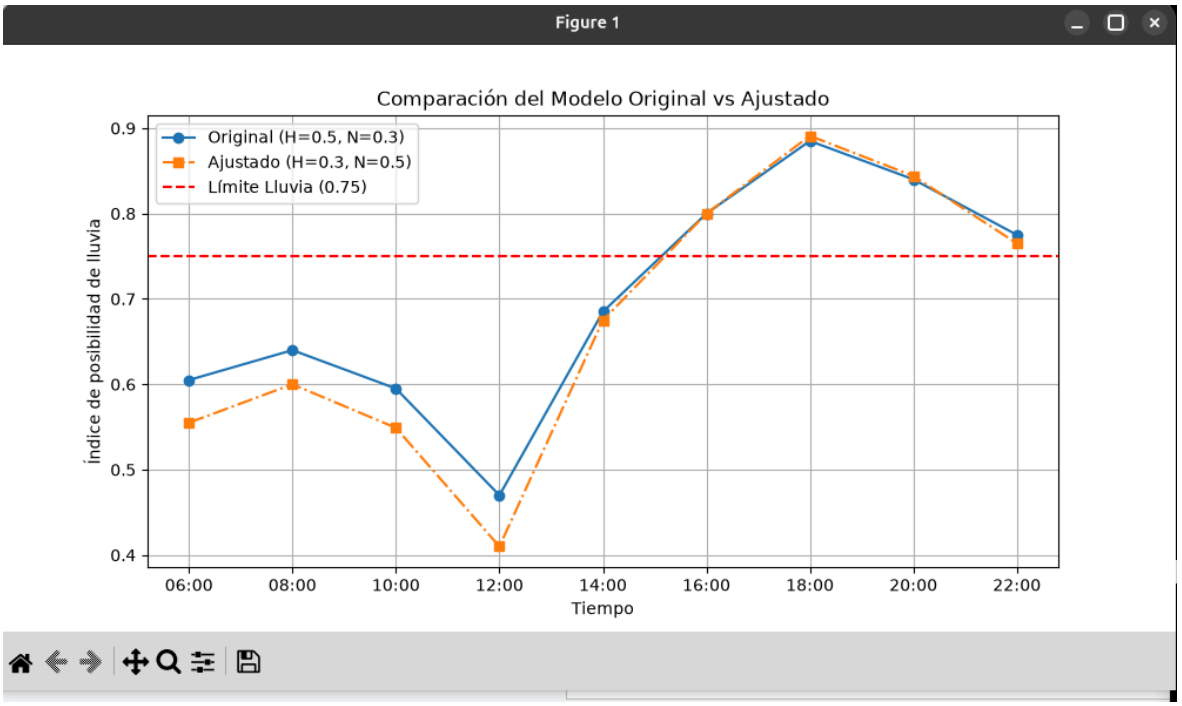
mod_original = ModeloClima(peso_h=0.5, peso_n=0.3, peso_tf=0.2)
mod_ajustado = ModeloClima(peso_h=0.3, peso_n=0.5, peso_tf=0.2)

mi_vista = VistaClima()

mi_controlador = ControladorClima(mod_original, mod_ajustado, mi_vista)

mi_controlador.iniciar_simulacion_completa(lista_horas, lista_humedad, lista_nubosidad, lista_temp)
```

- Gráficas de crecimiento y decrecimiento.



- Identificación de la influencia de r.
- Interpretación matemática del comportamiento

```
miguel_15@miguel-loq: ~/PycharmProjects/Simulacion_Practica1 — python3 ma...
~/PycharmProjects/Simulacion_Practica1

miguel_15@miguel-loq:~/PycharmProjects/Simulacion_Practica1$ source venv/bin/activate
(venv) miguel_15@miguel-loq:~/PycharmProjects/Simulacion_Practica1$ python3 main.py

================ MODELO ORIGINAL ================
Hora   | H    | N    | Tf   | Indice | Estado
-------------------------------------------------------
06:00  | 0.65 | 0.40 | 0.80 | 0.605  | Lluvia probable
08:00  | 0.70 | 0.50 | 0.70 | 0.640  | Lluvia probable
10:00  | 0.68 | 0.45 | 0.60 | 0.595  | Baja posibilidad
12:00  | 0.60 | 0.30 | 0.40 | 0.470  | Baja posibilidad
14:00  | 0.75 | 0.70 | 0.50 | 0.685  | Lluvia probable
16:00  | 0.85 | 0.85 | 0.60 | 0.800  | Lluvia
18:00  | 0.92 | 0.95 | 0.70 | 0.885  | Lluvia
20:00  | 0.88 | 0.90 | 0.65 | 0.840  | Lluvia
22:00  | 0.80 | 0.75 | 0.75 | 0.775  | Lluvia

================ MODELO AJUSTADO ================
Hora   | H    | N    | Tf   | Indice | Estado
-------------------------------------------------------
06:00  | 0.65 | 0.40 | 0.80 | 0.555  | Baja posibilidad
08:00  | 0.70 | 0.50 | 0.70 | 0.600  | Lluvia probable
10:00  | 0.68 | 0.45 | 0.60 | 0.549  | Baja posibilidad
12:00  | 0.60 | 0.30 | 0.40 | 0.410  | Baja posibilidad
14:00  | 0.75 | 0.70 | 0.50 | 0.675  | Lluvia probable
16:00  | 0.85 | 0.85 | 0.60 | 0.800  | Lluvia
18:00  | 0.92 | 0.95 | 0.70 | 0.891  | Lluvia
20:00  | 0.88 | 0.90 | 0.65 | 0.844  | Lluvia
22:00  | 0.80 | 0.75 | 0.75 | 0.765  | Lluvia
S
```

7. Preguntas de Control:

1. ¿Qué es un modelo matemático?

Es una representación simplificada y abstracta de un sistema o experimento del mundo real. Aquí, los resultados o elementos del mundo físico se traducen en objetos y estructuras matemáticas despojando las propiedades no esenciales para enfocar el análisis únicamente en los aspectos relevantes del fenómeno

2. ¿Cuál es la diferencia entre variable y parámetro?

Parámetro: Es una constante fija que describe el escenario o las condiciones del mundo modelado.Los parámetros se eligen antes de ejecutar la simulación mediante un proceso conocido como parametrización y no cambian a lo largo de una corrida específica, pero sus valores pueden modificarse de manera sistemática entre diferentes corridas mediante un barrido de parámetros.

Variable: Es una propiedad o cantidad cuyo valor cambia dinámicamente a lo largo de la ejecución del modelo o cuyo resultado depende del desarrollo del experimento y se encargan de registrar y reflejar la evolución del modelo en el tiempo.

3. ¿Qué representa P_0?

Representa la probabilidad de que el estado del sistema sea cero o la probabilidad de que ocurran cero eventos.

4. ¿Qué ocurre cuando r < 0?

Ocurre una invalidez de dominio (o *tasa no física*): en procesos estocásticos y de simulación, la tasa r (o λ) exige ser estrictamente positiva ($r > 0$), por lo que un valor negativo destruye la definición de la función de densidad de probabilidad y genera un error de ejecución

5. ¿Qué limitaciones tiene el modelo exponencial?

La falta de memoria es su propiedad matemática definitoria, la cual asume que la probabilidad de ocurrencia futura es totalmente independiente del tiempo ya transcurrido, ignorando el desgaste real de los componentes o sistemas.

8. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	
Aprobado por	Edison L Coronel Romero **Director de Carrera**	